

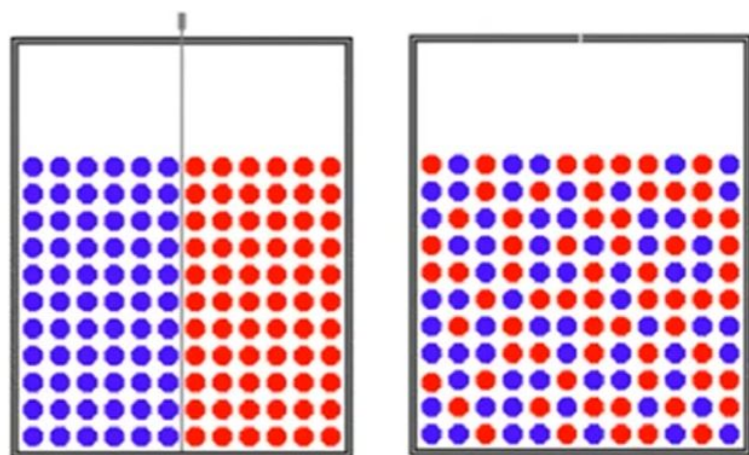
Refactorización: mejora del código



Entropía creciente

Todo código va empeorando su calidad con el tiempo

=> entropía, degradación



Refactorizaciones

Mejorar código, haciéndolo más comprensible
Sin cambiar funcionalidad

Refactorización

“Refactoring”

Mejorar el código ya escrito

¿Cómo?

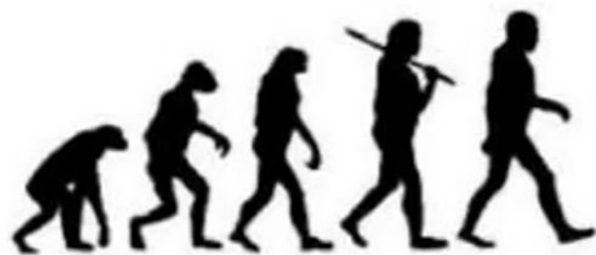
Modificar estructura interna

Sin modificar comportamiento observable

Ejemplos:

Eliminar código duplicado

Introducir polimorfismo



Refactoring
Improving the Design of Existing Code

¿Hacemos un ejemplo?

CajaAhorro >> extraer(monto)

```
if (monto <= 0)
    throw new MontoInvalido();
if (monto > this.getSaldo())
    throw new SaldoInsuficiente();
this.saldo -= monto;
```

CuentaCorriente >> extraer(monto)

```
if (monto <= 0)
    throw new MontoInvalido();
if (monto > this.getSaldo() + this.descubierto)
    throw new SaldoInsuficiente();
this.saldo -= monto;
```



Para qué

Mejorar código, haciéndolo más comprensible

- Para modificaciones

- Para depuraciones

- Para optimizaciones

Mantener alta la calidad del código

- Si no, se degrada

A la larga, aumenta la productividad



Cuándo refactorizamos

Actitud constante

Consecuencia de revisiones de código

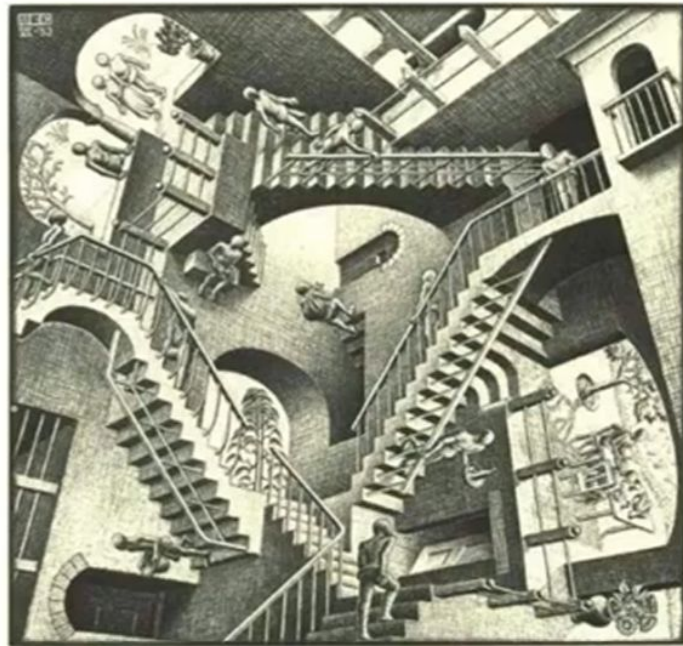
Antes de modificar código existente

Después de incorporar funcionalidad

Antes de optimizar

Ojo: optimizar $\neq$ refactorizar

Durante depuraciones



Condiciones previas

Riesgo alto

Máxima: “Si funciona, no lo arregle”

Un paso por vez

Pruebas automatizadas

Escribirlas antes de refactorizar

Y correrlas luego de cada pequeño cambio



Problemas y refactorización

“Bad smells in code” (malos olores), los llama Fowler

Son indicadores de que algo está mal, y se solucionan con refactorizaciones

Hay catálogos

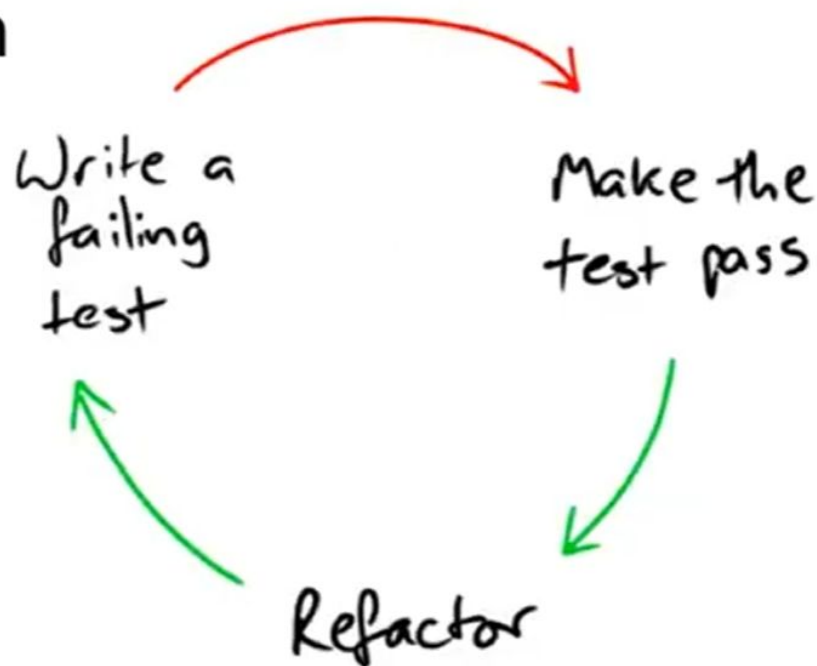
Smalltalk desde los inicios

Java: lenguaje habitual



TDD: recapitulación

Test-Driven Development =
Test-First +
Automatización +
Refactorización

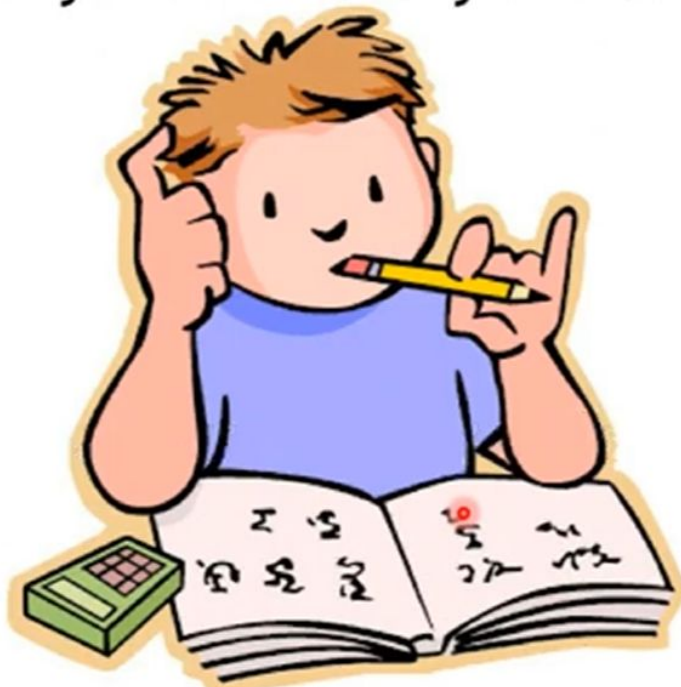


Recapitulación: preguntas

¿Para qué refactorizamos?

¿Por qué no hacemos varias refactorizaciones seguidas?

¿Qué relación hay entre TDD y refactorización?



Claves

Polimorfismo = distintos comportamientos para un mismo mensaje

Polimorfismo seguro y sin herencia: interfaces

Manejar la entropía => refactorización



Lectura obligatoria

“Replace Conditional with Polymorphism”,
<https://sourcemaking.com/refactoring/replace-conditional-with-polymorphism>



Qué sigue

Profundización

UML

Excepciones

Otros

Calidad de código

